Assignment 3 - Refactoring Report

Luka Dundjerovic | 200494589

ENSE 370 - Software System Design

March 29, 2026

Table of Contents

Introduction.....3
Initial System Understanding.....3
Code Smells Identified.....4
Refactoring Plan.....9
Step-by-Step Refactoring Evidence.....14
Final Design Summary.....14
Reflection.....14

GitHub Link to Assignment

https://github.com/LukaD-77/Assignment3-Refactoring-370/tree/main

Introduction

This report documents the step-by-step refactoring of a legacy University Registration System developed in Java. The original application successfully simulates a university environment by handling core operations such as student enrollment, course management, grading, payment processing, and transcript generation. While the system is functionally operational, its underlying architecture suffers from significant design flaws, including tight coupling, mixed responsibilities, and poor encapsulation.

The primary objective of this assignment is to analyze the starter code using IntelliJ to identify common code smells and systematically refactor the application without altering its intended behavior. Ultimately, this process aims to transform the codebase into a more cohesive, readable, and maintainable state.

The following sections will detail the initial assessment of the system, outline the identified code smells, and present a structured refactoring plan. Furthermore, this report provides chronological evidence of the refactoring process and concludes with an analysis of the final system design and a reflection on the decisions made throughout the project.

Initial System Understanding

The legacy system simulates university administration by tracking students, courses, enrollments, instructors, and financial records. Currently, the architecture suffers from poor separation of concerns. The Main class handles both application flow and heavy UI input, utilizing long, unreadable conditional chains for its menu system.

Meanwhile, UniversitySystem functions as a tightly coupled "God Class" that manages everything from core business rules (grading, enrollment) to presentation logic (printing

transcripts and summaries). The remaining domain classes strictly define data fields but lack proper encapsulation.

The table below summarizes the files included in the starter package, their purpose, and their existing structural issues:

File	Purpose	Notes
Main.java	Interactive console entry point	Mixes input handling and application flow.
UniversitySystem.java	Main legacy manager class	God class with duplicated logic and long methods.
Student.java	Student model	Public fields and weak encapsulation.
Course.java	Course model	Use primitive strings and public state.
Enrollment.java	Enrollment model	Stores raw strings for grade and schedule.
Instructor.java	Instructor model	Underused and weakly modeled.
PaymentRecord.java	Payment model	Simple record with public fields.
AdminHelper.java	Legacy helper functions	Partly useful, partly inconsistent.
LegacyReportPrinter.java	Output/reporting class	Adds more design opportunities.
AuditLogger.java	Logging helper	It can be integrated or redesigned later.

Table 1: *Files Included in Starter Package. From Assignment 3 Refactoring Manual*

Code Smells Identified

This section will outline some of the major code smells found in the legacy system. The changes will mainly be outlined in Main.java and UniversitySystem.java.

Main.java:

1) Long Method

The main method is over 140 lines of code. The logic for each menu item could be split up into its own helper method.

```java
public class Main {  Luka Dunjerovic
    public static void main(String[] args) {  Luka Dunjerovic
        Scanner scanner = new Scanner(System.in);
        UniversitySystem system = new UniversitySystem();

        while (true) {
            System.out.println("\n===== UNIVERSITY SYSTEM =====");
            System.out.println("1. Add Student");
            System.out.println("2. Add Course");
            System.out.println("3. Add Instructor");
            System.out.println("4. Enroll Student");
            System.out.println("5. Assign Grade");
            System.out.println("6. Process Payment");
            System.out.println("7. View Transcript");
            System.out.println("8. View Course Roster");
            System.out.println("9. View Department Summary");
            System.out.println("10. Send Warning Letters");
            System.out.println("11. View All Students");
            System.out.println("12. View All Courses");
            System.out.println("13. View All Payments");
            System.out.println("14. Exit");
            System.out.print("Choose option: ");

            int choice;
            try {
                choice = Integer.parseInt(scanner.nextLine());
            } catch (Exception ex) {
                System.out.println("Invalid input. Please enter a number.");
                continue;
            }

            if (choice == 1) {
```

2) Switch Statement Overuse

The conditional if statements spans 14 different options in the main function.

```java
public class Main {  Luka Dunjerovic
    public static void main(String[] args) {  Luka Dunjerovic

            } else if (choice == 9) {
                System.out.print("Enter Department Code (e.g., CS, SE, IT): ");
                String dept = scanner.nextLine();
                system.printDepartmentSummary(dept);

            } else if (choice == 10) {
                system.sendWarningLetters();

            } else if (choice == 11) {
                LegacyReportPrinter printer = new LegacyReportPrinter();
                printer.printStudents(system.students);

            } else if (choice == 12) {
                LegacyReportPrinter printer = new LegacyReportPrinter();
                printer.printCourses(system.courses);

            } else if (choice == 13) {
                LegacyReportPrinter printer = new LegacyReportPrinter();
                printer.printPayments(system.payments);

            } else if (choice == 14) {
                System.out.println("Exiting system...");
                break;

            } else {
                System.out.println("Invalid option.");
            }
        }
```

3) Mixed Responsibilities

The main method is responsible for printing, reading user input, and instantiating complex objects. The printing can be split up into multiple helper methods in the main function. Each individual class should be responsible for instantiating objects and accessing data. The main should only change the data through setters and getters.

4) Redundant Code

The LegacyReportPrinter item is redeclared in multiple conditional statements. This could be instantiated once earlier in the program to avoid redundancy.

```
} else if (choice == 11) {
    LegacyReportPrinter printer = new LegacyReportPrinter();
    printer.printStudents(system.students);

} else if (choice == 12) {
    LegacyReportPrinter printer = new LegacyReportPrinter();
    printer.printCourses(system.courses);

} else if (choice == 13) {
    LegacyReportPrinter printer = new LegacyReportPrinter();
    printer.printPayments(system.payments);
```

5) Confusing Variable Names

There are certain variable names that are not clear in the main method. These include: sid, ccode, sem, method, dept. These should be renamed to studentID, courseCode, semester, paymentMethod, and department.

UniversitySystem.java:

1) Large Class

This class is a “God Class” and has too many responsibilities. It handles logic for enrollment, assigning grades, processing payments, sending warnings, and printing information.

2) Long Methods

The methods enrollStudent, assignGrade, and processPayment are all long methods.

```java
public class UniversitySystem {  2 usages  Luka Dunjerovic *
    public void enrollStudent(String studentId, String courseCode, String semester, String paymentType) {  1 usage  Luka Dunjerovic
        Student s = null;
        Course c = null;

        for (Student st : students) {
            if (st.id.equals(studentId)) {
                s = st;
            }
        }

        for (Course co : courses) {
            if (co.code.equals(courseCode)) {
                c = co;
            }
        }

        if (s == null) {
            System.out.println("Student not found");
            logs.add("Student not found: " + studentId);
            return;
        }

        if (c == null) {
            System.out.println("Course not found");
            logs.add("Course not found: " + courseCode);
            return;
        }
```

3) Duplicate Code

There are repeated for loops to find a specific student and course in enrollstudent(), assignGrade(), processPayment(), and printTranscript. These are split into their own helper functions at the bottom of the class, but are never used.

```java
public void enrollStudent(String studentId, String courseCode, String semester, String paymentType) {  1 usage  Luka Dunjerovic
    Student s = null;
    Course c = null;

    for (Student st : students) {
        if (st.id.equals(studentId)) {
            s = st;
        }
    }

    for (Course co : courses) {
        if (co.code.equals(courseCode)) {
            c = co;
        }
    }
```

There is a conditional that checks if the student is null that is repeated in three different methods.

```java
if (s == null) {
    System.out.println("Student not found");
    logs.add("Student not found: " + studentId);
    return;
}
```

4) Magic numbers

The rates for different types of students are declared at the start of the class, but never used and not declared as constants. Later in the code these same values are hard coded into the methods. The

5) Magic Strings

The student types of LOCAL, INTERNATIONAL, and SCHOLARSHIP are hard coded into multiple methods. The payment methods of INSTALLMENT, CARD, CASH, and BANK are also not defined as constants and hard coded into methods. Student status of PROBATION, GOOD, and HONOR are hard coded. Grades of A, B, C, D, F are hard coded.

6) Inappropriate Intimacy / Weak Encapsulation

The class depends heavily on the details of other classes in the system. Variable declarations inside of the class should be declared as private.

7) Confusing Variable Names

The single letter variables like s, c, and e are used throughout the program and can make it confusing to read. These could be changed to more appropriate names such as, student, course, and enrollment.

```
public void enrollStudent(String studentId, String courseCode, String semester, String paymentType) {  1 usage  Luka Dunjerovic
    Student s = null;
    Course c = null;
```

Student, PaymentRecord, Instructor, Enrollment, Course:

1) Weak Encapsulation / Data Class

All of these classes practice poor encapsulation because all of their fields are public and they don't have any setters and getters. Other classes should not be able to directly change the data of an object. Additionally, these classes don't have any methods, they only have constructors. We

can improve encapsulation by making the fields private and allowing other classes to access the data through setters and getters. Functionality specific to each class can be moved from the university system logic to each individual class.

AdminHelper.java:

1) Dead Code

None of the code in this class is ever used throughout the entire system.

```
public static boolean validEmail(String email) {  no usages  Luka Dunjerovic
    if (email == null) {
        return false;
    }
    if (email.contains("@") && email.contains(".")) {
        return true;
    }
    if (email.contains("@")) {
        return true;
    }
    return false;
}

public static double calculateLatePenalty(double balance) {  no usages  Luka Dunjerovic
    if (balance > 1000) {
        return 100;
    } else if (balance > 500) {
        return 50;
    } else if (balance > 200) {
        return 20;
    }
    return 0;
}

public static String academicRemark(double gpa) {  no usages  Luka Dunjerovic
    if (gpa >= 3.5) {
        return "Excellent";
    } else if (gpa >= 2.0) {
        return "Satisfactory";
    } else {
```

Refactoring Plan

Stage 1: Remove Dead Code & Variable Renaming

Problems Addressed:

- Confusing Variable Names (Main.java): Variables like sid, ccode, sem, method, and dept lack clarity.
- Confusing Variable Names (UniversitySystem.java): Single-letter variables like s, c, and e are used throughout, reducing readability.
- Dead Code (AdminHelper.java): None of the code in this class is ever used.

Refactoring Techniques: Rename Variable, Remove Redundancy.

Expected Benefit:

- Improves code readability. Removes unnecessary files that clutter the project.

Associated Risk:

- Very low. Could make code less understandable if poor names are given.

Stage 2: Improve Domain Design (Magic Values)

Problems Addressed:

- Magic Numbers (UniversitySystem.java): Rates for different student types are declared at the top of the class but ignored. Values like 300, 550, 100 are hard-coded directly into the logic.
- Magic Strings (UniversitySystem.java): Student types, payment methods, statuses , and grades are all hard-coded strings.

Refactoring Techniques:

- Replace Magic Numbers with Constants, Replace Primitive with Enum/Constant.

Expected Benefit:

- Centralizes configuration values and prevents silent bugs caused by string typos, enforcing compile-time type safety.

Associated Risk:

- Low.

Stage 3: Remove Duplication

Problems Addressed:

- Redundant Code (Main.java): LegacyReportPrinter is instantiated multiple times in different conditional branches.
- Duplicate Code (UniversitySystem.java): Repeated for loops to find specific students and courses, and repeated null checks across three different methods. The provided helper functions at the bottom of the class are completely ignored.

Refactoring Techniques:

- Remove Duplicate Code, Reuse Method (utilizing findStudent and findCourse).

Expected Benefit:

- Centralizes logic. Future updates will only need to happen in one place, preventing systemic inconsistencies and shortening the code.

Associated Risk:

- Low. Replacing duplicated inline loops with existing method calls is generally a safe operation.

Stage 4: Improve Encapsulation

Problems Addressed:

- Weak Encapsulation / Data Class (Student, PaymentRecord, Instructor, Enrollment, Course): All fields are public, and the classes lack getters, setters, or behavioral methods.
- Inappropriate Intimacy (UniversitySystem.java): The class depends heavily on the internal details of other classes and directly modifies their public fields.

Refactoring Techniques:

- Encapsulate Field (change to private, generate getters/setters), Move Method (shift specific logic into the respective domain models).

Expected Benefit:

- Protects object data integrity. Classes will interact through defined methods rather than directly altering each other's states.

Associated Risk:

- Medium to High. Changing field visibility will temporarily break existing code across Main and UniversitySystem, requiring widespread updates from direct field access (e.g., s.name) to method calls (e.g., s.getName()).

Stage 5: Method Extraction

Problems Addressed:

- Long Method (Main.java): The main method is over 140 lines long.
- Long Methods (UniversitySystem.java): Methods like enrollStudent, assignGrade, and processPayment try to accomplish too many tasks at once.

Refactoring Techniques:

- Extract Method, Decompose Conditional.

Expected Benefit:

- Breaks down massive execution blocks and 14-branch conditionals into small, single-purpose helper methods (e.g., handleEnrollment(), calculateFee()), drastically reducing cognitive load and complexity.

Associated Risk:

- Medium. Extracting logic requires careful attention to variable scoping and parameter passing.

Stage 6: Separate Concerns

Problems Addressed:

- Mixed Responsibilities (Main.java): The main method handles UI printing, reading user input, and instantiating complex objects directly.

Refactoring Techniques:

- Separate Presentation Logic from Business Logic, Move Method.

Expected Benefit:

- Decouples the UI from the underlying logic. The Main class will only act as an interface, delegating object creation and data mutation to the system's setter methods and builders.

Associated Risk:

- Medium. Requires rerouting how data is passed from the Scanner to the core system.

Stage 7: Structural Refactoring

Problems Addressed:

- Large Class / God Class (UniversitySystem.java): This class currently handles enrollment, grading, payments, warnings, and UI printing all by itself.

Refactoring Techniques:

- Extract Class (e.g., EnrollmentService, PaymentService, ReportingService).

Expected Benefit:

- Achieves a highly cohesive, service-oriented architecture that strictly follows the Single Responsibility Principle.

Associated Risk:

- High. Moving substantial chunks of core logic into entirely new files can break dependencies. This stage is intentionally placed near the end so it operates on thoroughly cleaned and encapsulated methods.

Step-by-Step Refactoring Evidence

Below is the commit history in my repository to show that refactoring was done sequentially, in the steps mentioned above in the refactoring plan.

```
* bbbe28c (HEAD -> main, origin/main) split up complex methods in enrollmentService
* d15755b separate data clumps into specific classes
* 6d2a1de restructure into services
* d949eb9 separate printing logic
* 59b3ce6 extract methods from larger methods
* e672f13 improve encapsulation of classes
* 354aae9 remove duplicate code and use helper methods
* 47f5e2c replace magic strings and numbers with constants
* 400a6f0 step 1: rename variables
* cfe53fc add screeshot 2 and gitignore
* 1006c11 add first screenshot
* dcfecb0 inital repo setup
```

Final Design Summary

The original codebase suffered from several code smells, including large classes, long methods, dead code, message chains, redundancies, and poor encapsulation. Overall, the initial design was highly coupled and had low cohesion. My primary goal was to properly separate the concerns of the Main and UniversitySystem classes.

My approach to refactoring was to start with small, low-risk changes before tackling larger, high-risk structural overhauls. I began by updating variable names, removing dead and duplicate code, and declaring constants for hard-coded values. Next, I moved on to a slightly higher-risk refactor: encapsulating the data models (Course, Enrollment, Instructor,

PaymentRecord, and Student). I achieved this by making all fields private and adding getter and setter methods. This step carried some risk because Main and UniversitySystem were previously accessing these fields directly, meaning I had to carefully replace all direct references with the new methods to ensure the program still functioned correctly.

Following these changes, I reviewed the Main and UniversitySystem classes to break down long methods into smaller helper functions, working toward the Single Responsibility Principle. In Main, I replaced the long conditional block with a switch statement and extracted the user input and print statements for each case into their own methods to improve readability. In the UniversitySystem class, I split the lengthy enrollStudent, assignGrade, and processPayment methods into smaller helper methods to make the core logic easier to follow. Finally, I migrated all remaining printing methods to the LegacyReportPrinter class to properly separate presentation concerns from business logic.

The final design utilizes three separate service classes to handle the core logic for enrolling students, sending notifications, and processing payments. A dedicated report printer class now handles displaying all important university data to the user. The original data classes were kept but are now properly encapsulated. The UniversitySystem class was transformed into a UniversityDatabase class, stripping away its logic so it is solely responsible for storing university data. With these changes, the Main class is now strictly responsible for program flow and user input.